

Closest Pair of Points in 2D

(ADA-2023, CSE222)

Diptapriyo Majumdar

February 2, 2023

Closest Pair of Points (2D)

Problem definition:

- **Input:** A set P of n points in 2D plane.
- **Goal:** Compute a pair of points $q, r \in P$ such that $\text{dist}(q, r)$ is the smallest among all such pairs in P .
(**Weaker variant:** Compute δ such that δ is the minimum distance between every pair of points in P .)

Some Questions:

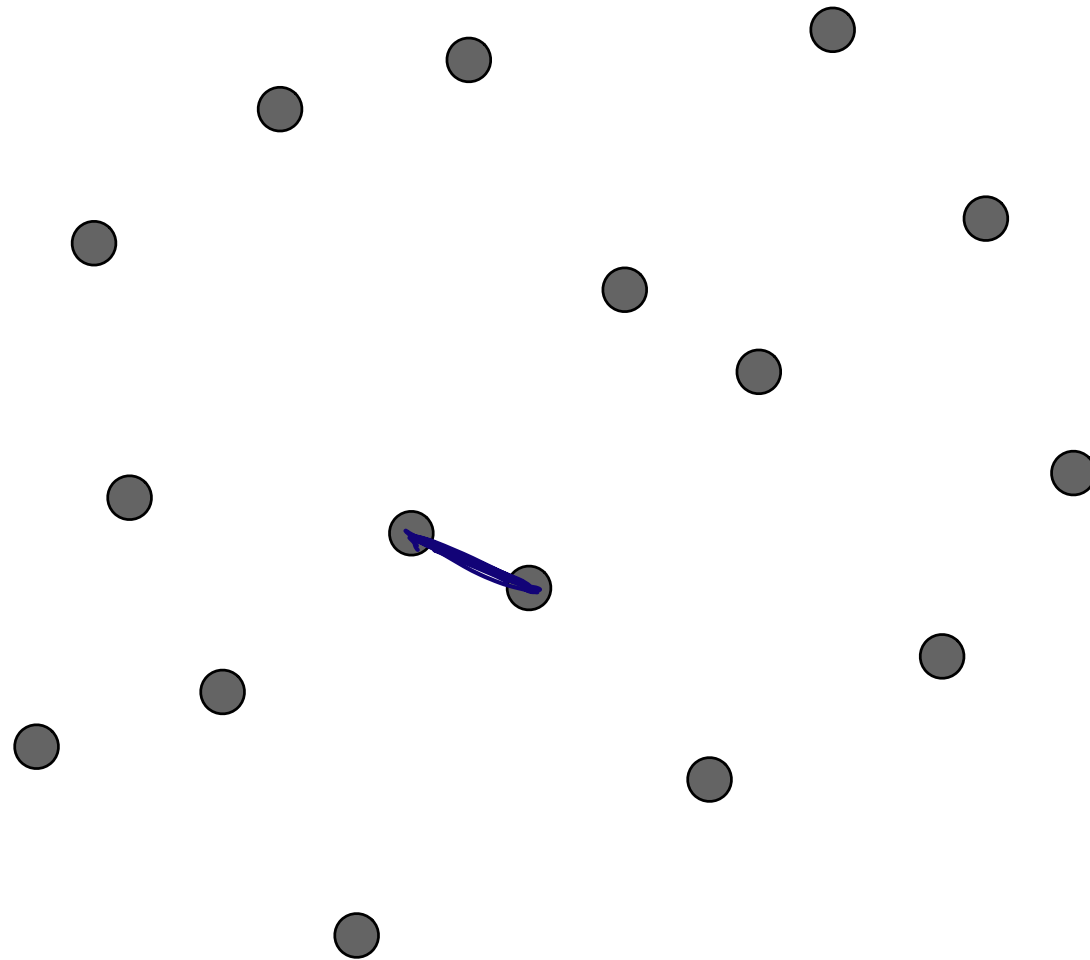
- What can be a very straightforward algorithm?

Compute the distance between all pair of points
 $\text{dist}(p_i, p_j)$ for all $p_i, p_j \in P$. $\binom{n}{2}$

Then find minimum of them.

Running time $O(n^2)$ -time.

Input Instance



1D makes simple?

Question: What if the input instance is in 1D?

There are n numbers. $a_1, a_2, \dots, a_n \in \mathbb{R}$

$|a_i - a_j|$ Find i, j ~~s.t.~~ such that

$|a_i - a_j|$ is smallest.

2, 10, 15, 25, 26, 36, 47, 64

1D makes simple?

Question: What if the input instance is in 1D?

- **Input:** There are n numbers. Find q, r from them such that $|q - r|$ is the smallest.
- **Question:** Can we solve this problem in $\underline{\underline{O(n \log_2 n)}}$ -time?

1. Sort the array. $\rightarrow O(n \log n)$ -time

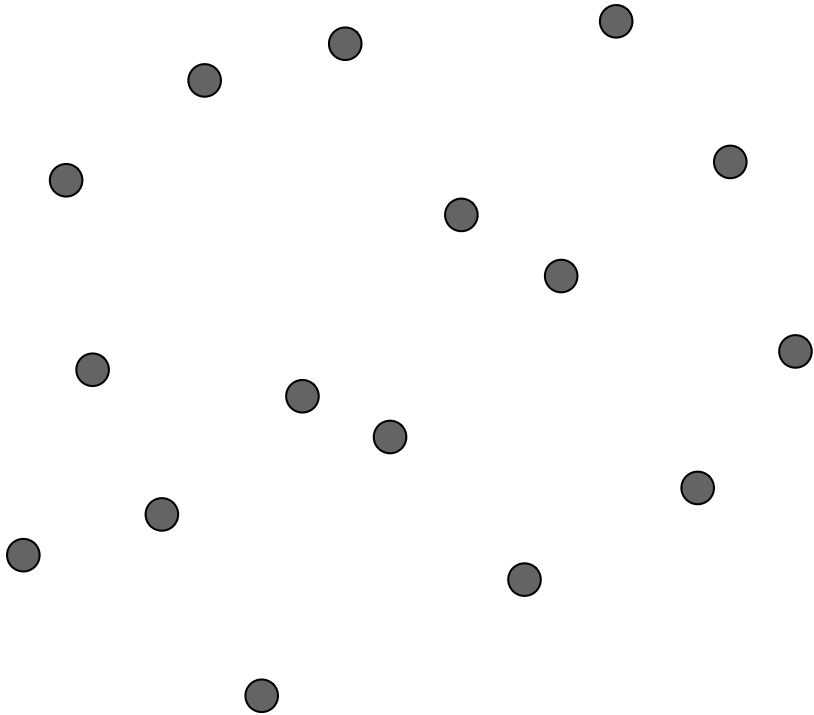
2. Check $|a_{i+1} - a_i|$ for all $i = 1, 2, \dots, n-1$.

Preprocessing

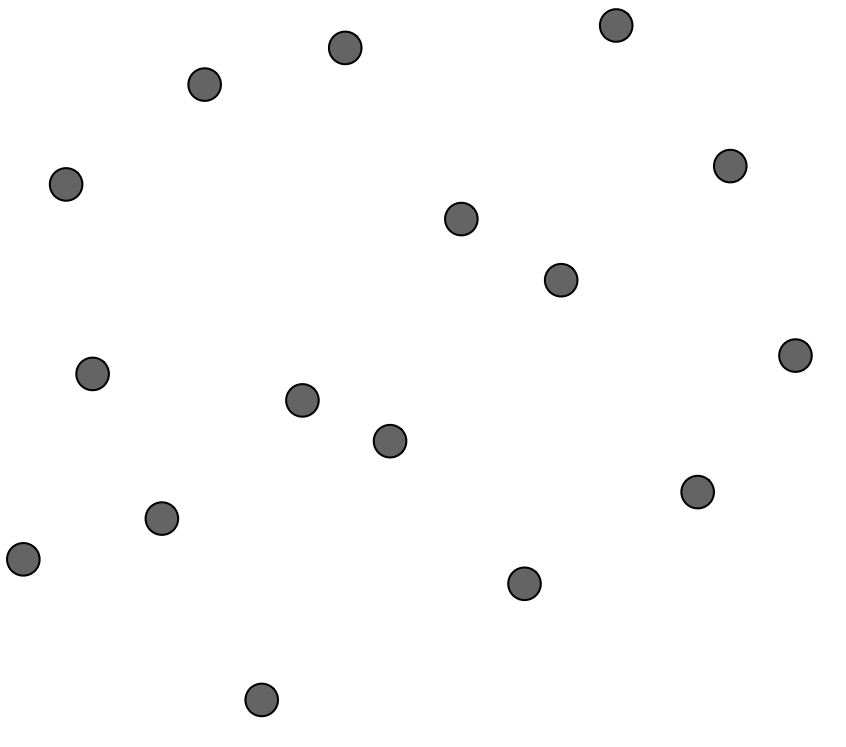
$\downarrow$
 $O(n)$ -time

Closest Pair in 2D

- **Input:** A set P of n points in 2D plane.
- **Goal:** Compute a pair of points $q, r \in P$ such that $\text{dist}(q, r)$ is the smallest among all such pairs in P .
(**Weaker version:** Compute δ such that δ is the minimum distance between every pair of points in P .)
- **Assumption:** All points have different q and r coordinates. The distance between two points, i.e. $\text{dist}(q, r)$ can be computed in $\mathcal{O}(1)$ -time.



Algorithm

- 
- It is unclear how we can divide the input points into two parts.

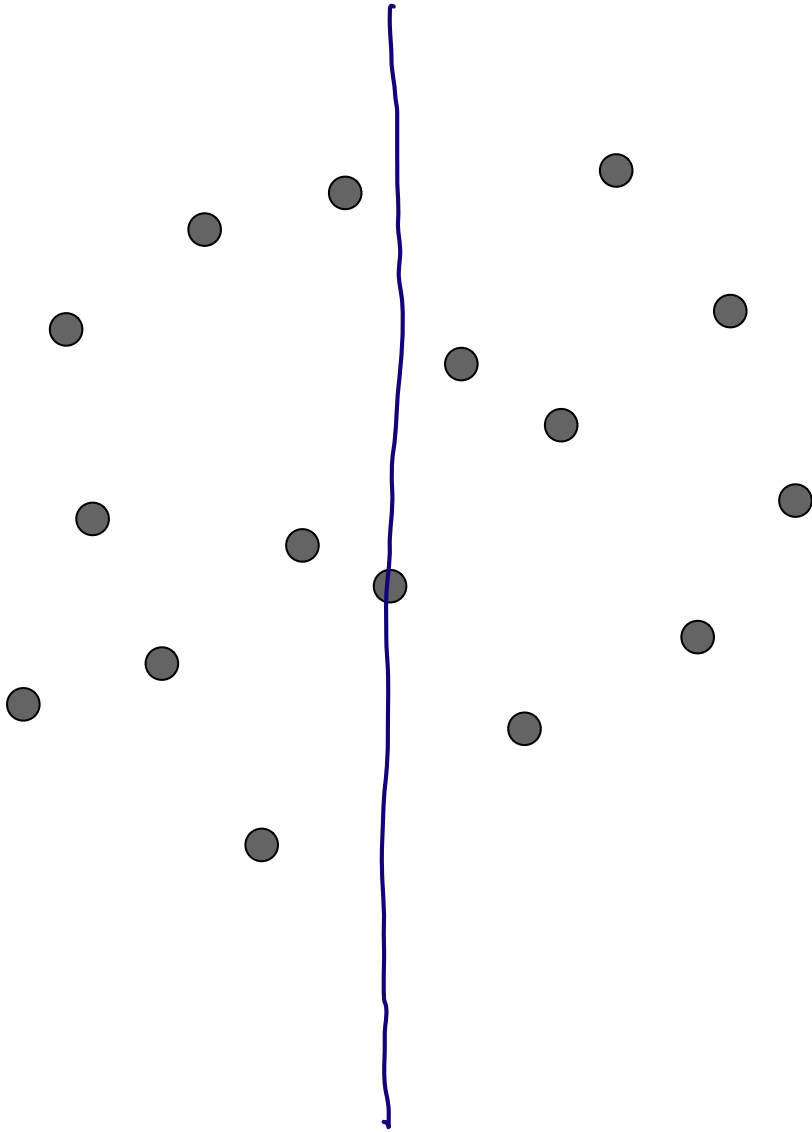
Sort them in increasing order of x coordinates

Sort them in increasing order of y coordinates

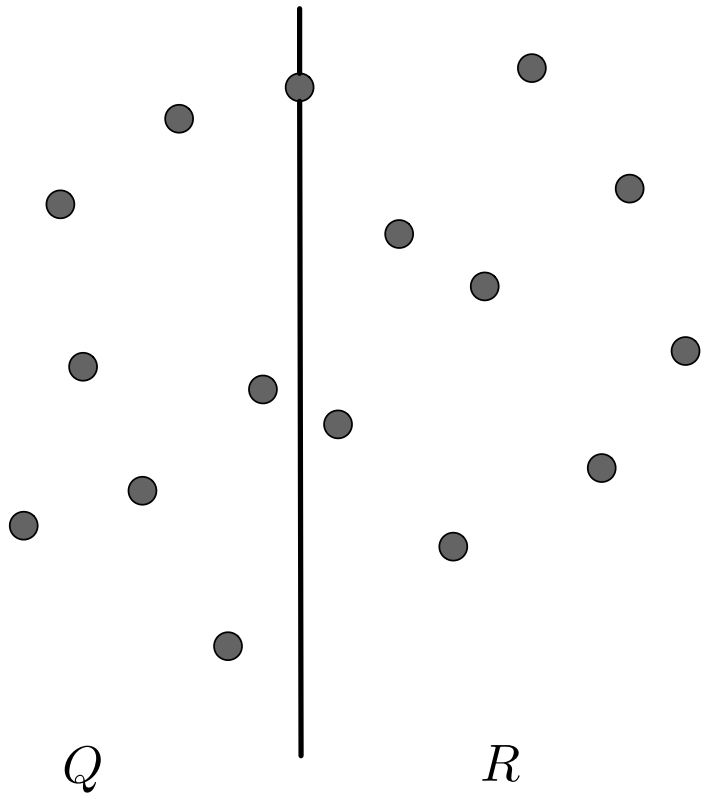
Algorithm

Input Set: P

- It is unclear how we can divide the input points into two parts.
- Sort the points based on x -coordinates.
Let P_x be the order.
- Sort the points based on y -coordinates.
Let P_y be the order.
- Then invoke a recursive algorithm.



Closest Pair in 2D

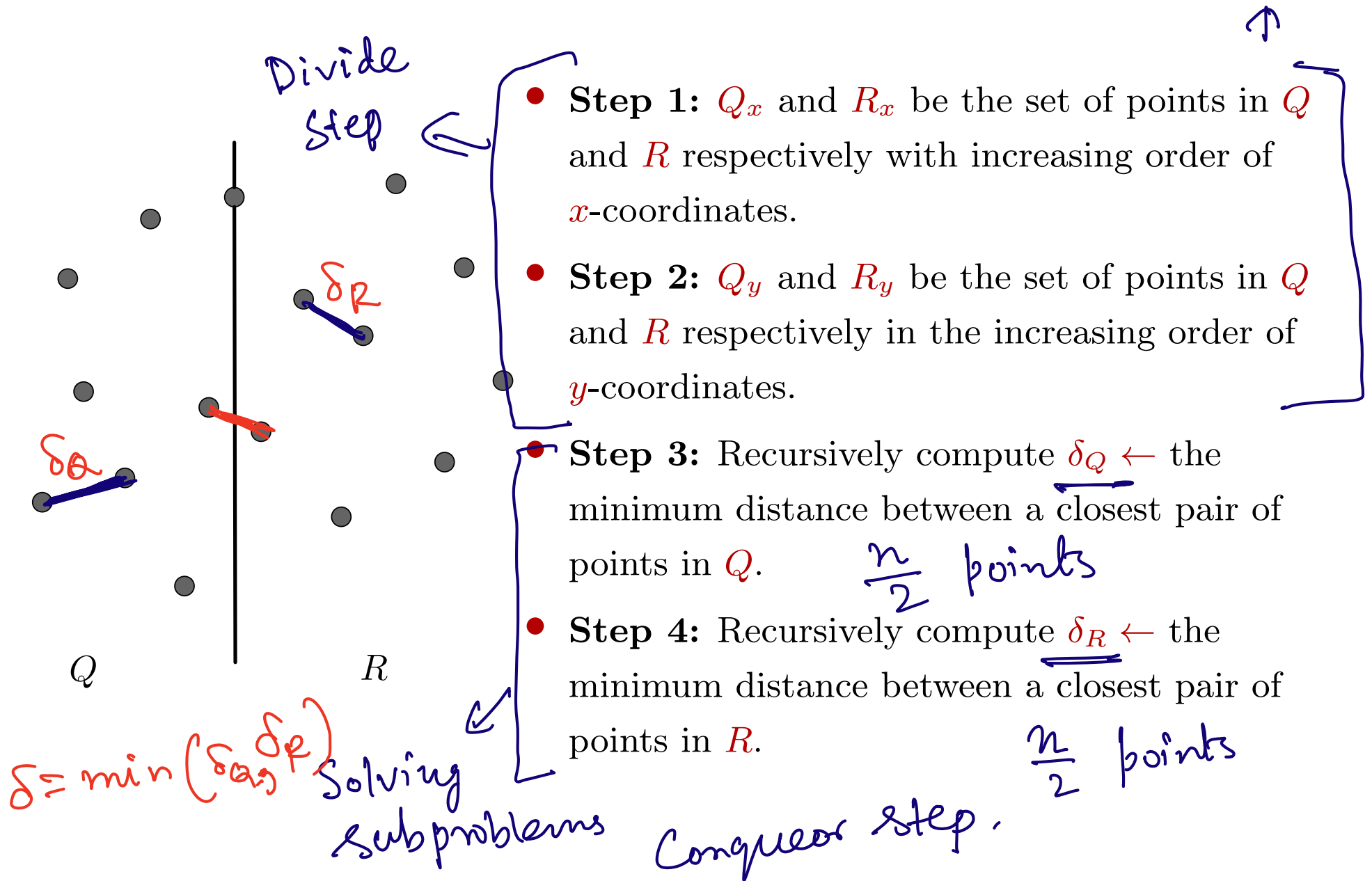


- **Step 1:** Q_x and R_x be the set of points in Q and R respectively with increasing order of x -coordinates.
- **Step 2:** Q_y and R_y be the set of points in Q and R respectively in the increasing order of y -coordinates.

Q_x the first $\frac{n}{2}$ points from P_x

R_x last $\frac{n}{2}$ points from P_x

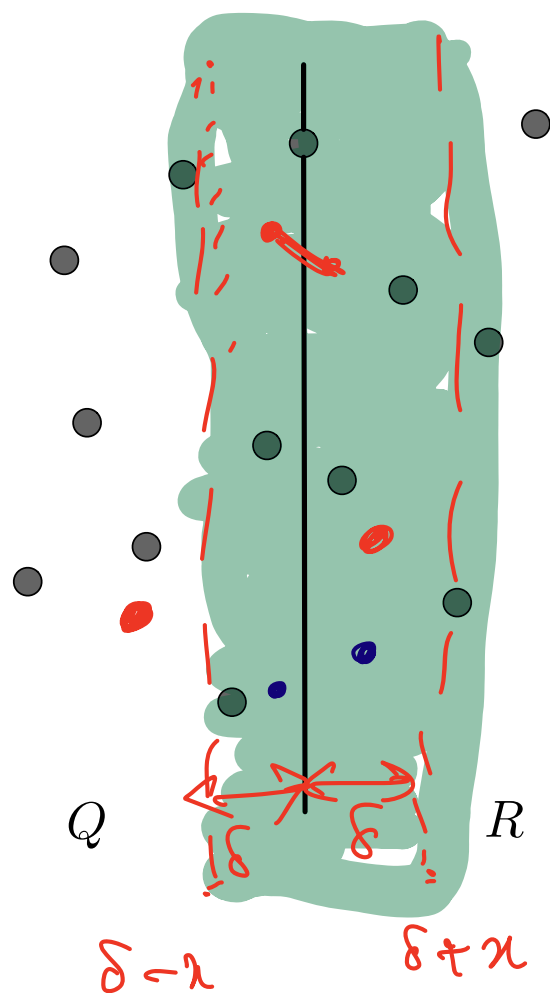
Closest Pair in 2D



$$\delta = \min\{\delta_Q, \delta_R\}$$

$$\hat{\delta} < \delta$$

Closest Pair in 2D



- **Step 1:** Q_x and R_x be the set of points in Q and R respectively with increasing order of x -coordinates.
- **Step 2:** Q_y and R_y be the set of points in Q and R respectively in the increasing order of y -coordinates. P_y
- **Step 3:** Recursively compute $\delta_Q \leftarrow$ the minimum distance between a closest pair of points in Q .
- **Step 4:** Recursively compute $\delta_R \leftarrow$ the minimum distance between a closest pair of points in R .

Combine $\left[\begin{array}{l} \text{Step 5: Combine the Information of } \delta_Q \text{ and } \\ \delta_R \text{ to compute the minimum distance } \delta \text{ in } P. \end{array} \right] \rightarrow \text{total}$

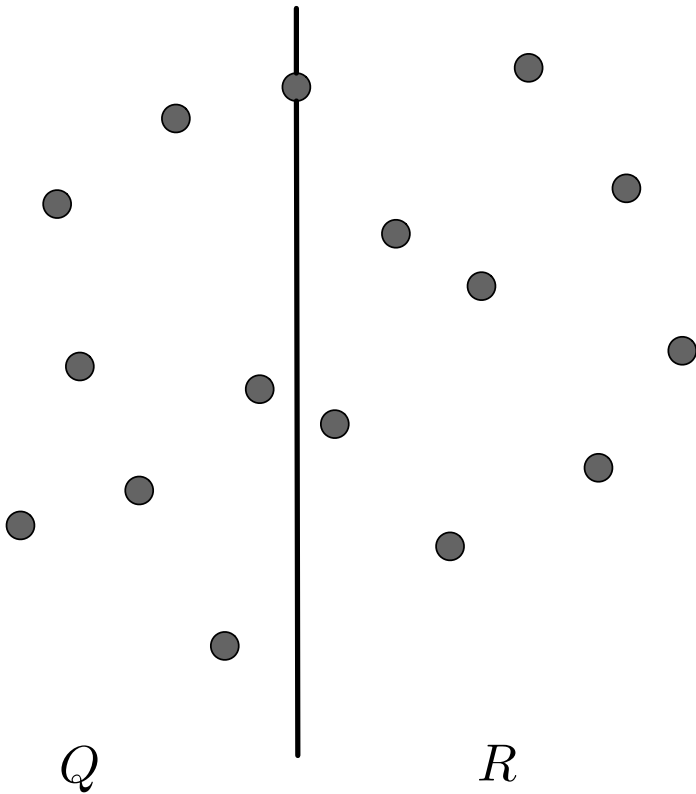
Main Algo

$P_x \leftarrow$ sort the points of P by
 x -coordinate;

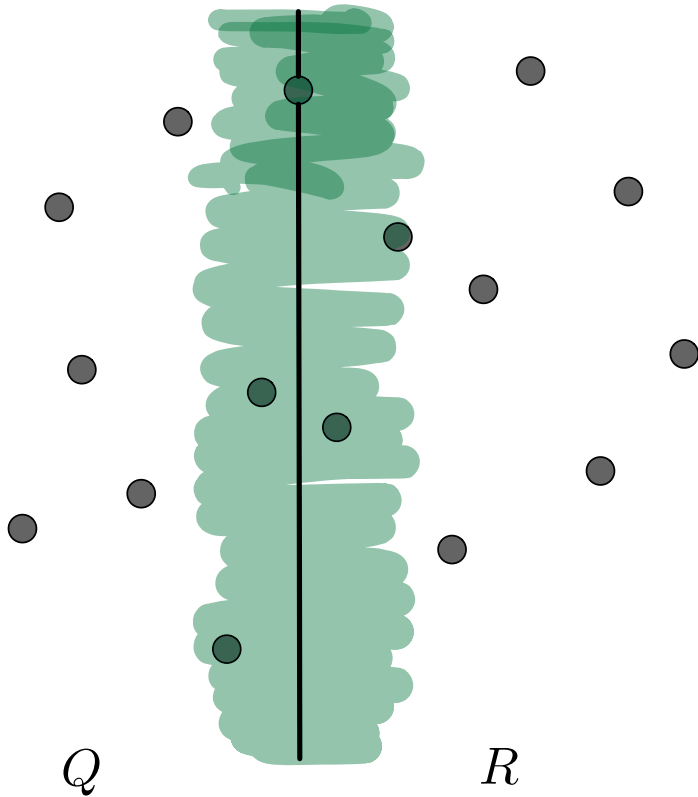
$P_y \leftarrow$ sort the points of P by
 y -coordinate;

$\rightarrow$ Preprocessing

$\delta \leftarrow \text{Closest-Pair-Rec}(P_x, P_y, n);$

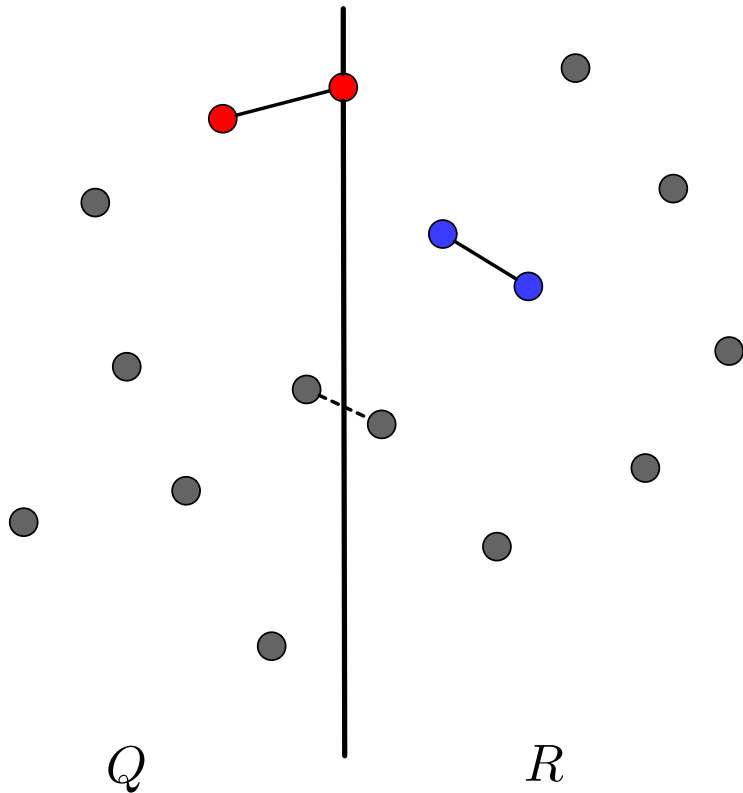


Closest Pair 2D



```
Closet-Pair-Rec( $P_x, P_y, n$ )
if  $|P| \leq 3$  then
    | Solve by checking distance between
    | all pairs;
end
else
    | Compute  $Q_x, R_x, Q_y, R_y$ ;
    |  $\delta_Q \leftarrow$ 
    | Closet-Pair-Rec( $Q_x, Q_y, \lceil n/2 \rceil$ );
    |  $\delta_R \leftarrow$ 
    | Closet-Pair-Rec( $R_x, R_y, \lfloor n/2 \rfloor$ );
    | Combine-Solutions-Q-and-R;
end
```

Closet Pair 2D

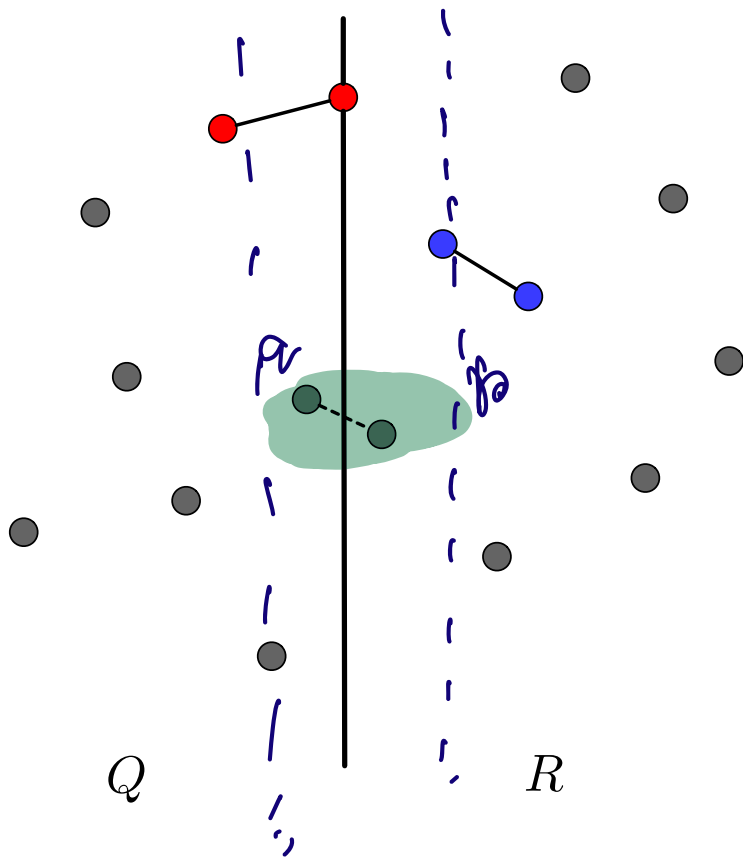


- How do we use the information of δ_Q and δ_R ?
- How can we use the information from sorting?
- In other words, how can we use the sorted order of $P_x, Q_x, R_x, P_y, Q_y, R_y$?
- If we try to check all pairs $(q, r) \in Q \times R$ and check $\text{dist}(q, r)$, then it takes $\mathcal{O}(n^2)$.

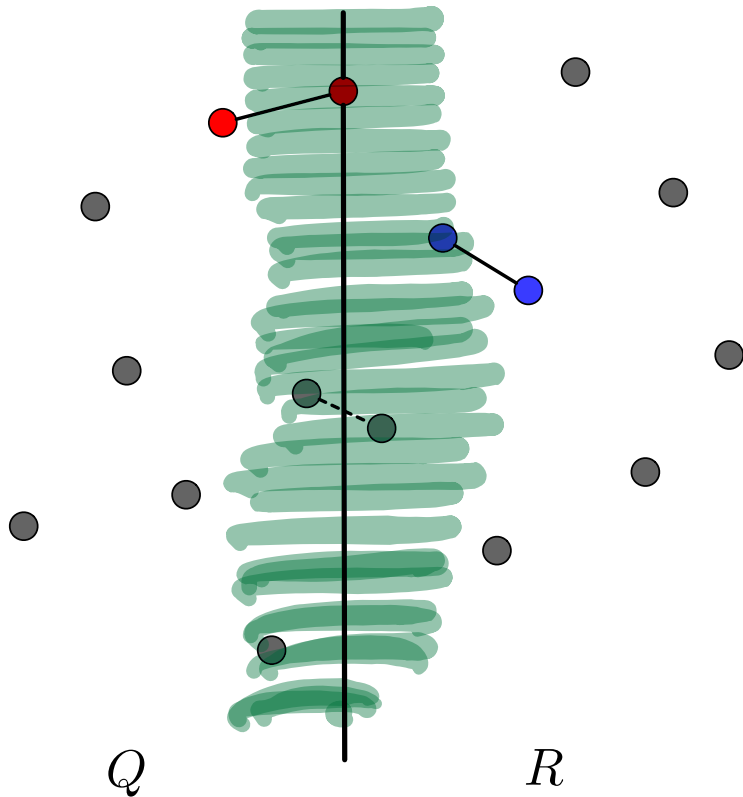
Checking all pairs of points
does not work.

Closest Pair

- Suppose that some $q \in Q$ and $r \in R$ is actually a witness for closest pair.
- It means that $\text{dist}(q, r) < \min\{\delta_Q, \delta_R\} =: \delta$
- How can q and r be located then?
- Let $\delta = \min\{\delta_Q, \delta_R\}$.

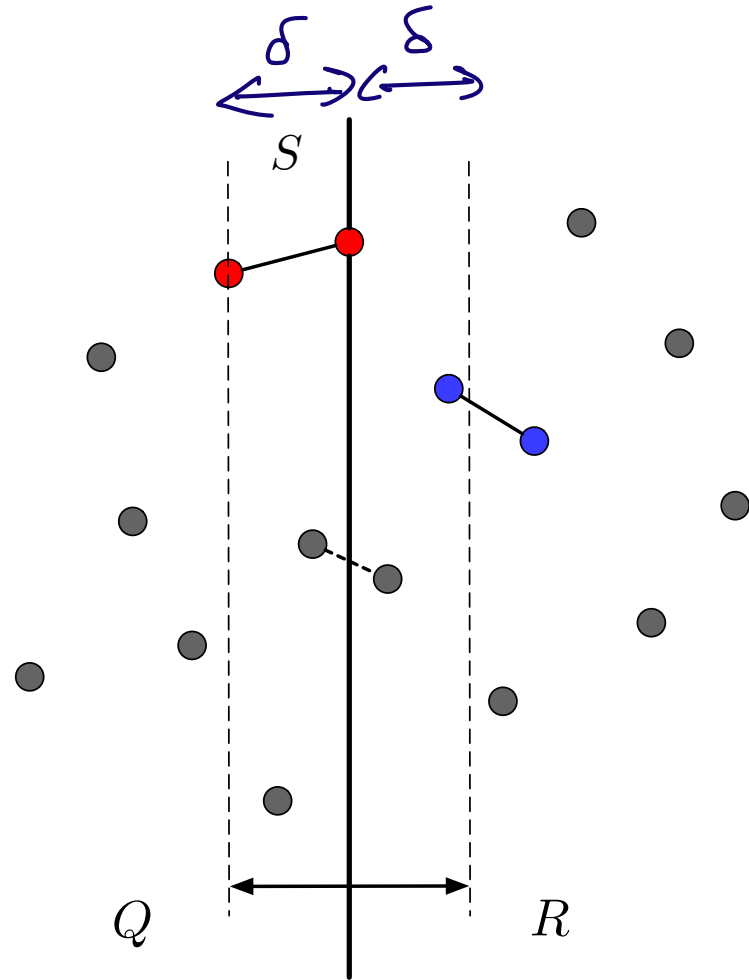


Closest Pair



- Suppose that some $q \in Q$ and $r \in R$ is actually a witness for closest pair.
- It means that $\text{dist}(q, r) < \min\{\delta_Q, \delta_R\}$.
- How can q and r be located then?
- Let $\delta = \min\{\delta_Q, \delta_R\}$.
- **Lemma:** If there exists $q \in Q$ and $r \in R$ for which $\text{dist}(q, r) < \delta$, then each of q and r lies within a distance of δ .

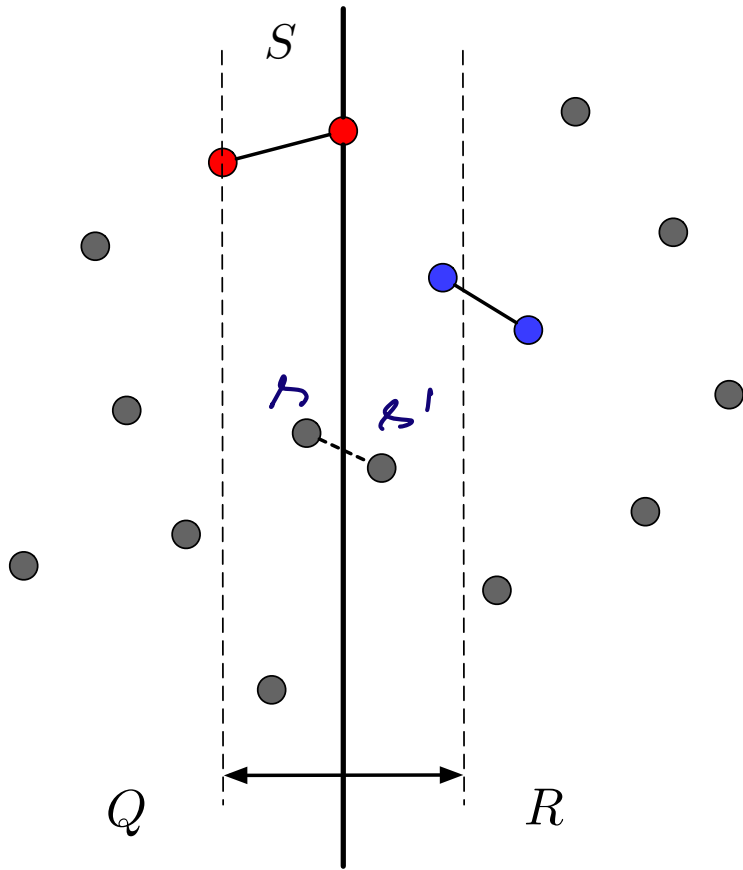
Closest Pair



- **Lemma 1:** If there exists $q \in Q$ and $r \in R$ for which $\text{dist}(q, r) < \delta$, then each of q and r lies within a distance of δ .
- It means that such a closest pair (q, r) must lie within this thin strip. Let S be this thin strip.
- Let S_y be the set of points of S in sorted order of y -coordinates.

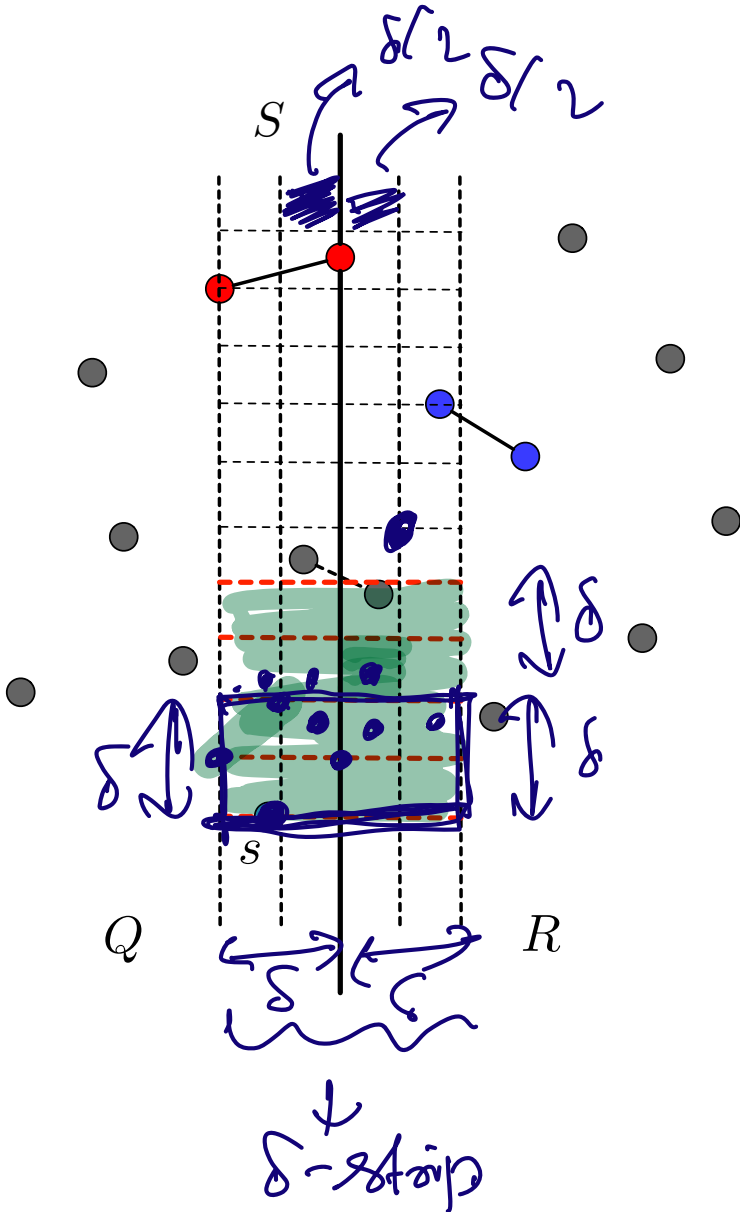
x -coordinates are at least

Closest Pair



- **Lemma 1:** If there exists $q \in Q$ and $r \in R$ for which $\text{dist}(q, r) < \delta$, then each of q and r lies within a distance of δ .
- It means that such a closest pair (q, r) must lie within this thin strip. Let S be this thin strip.
- Let S_y be the set of points of S in sorted order of y -coordinates.
- **Lemma 2:** There exist $q \in Q$ and $r \in R$ for which $\text{dist}(q, r) < \delta$ if and only if there exist $s, s' \in S$ such that $\text{dist}(s, s') < \delta$.

Closest Pair 2D



- Lemma 2:** There exist $q \in Q$ and $r \in R$ for which $\text{dist}(q, r) < \delta$ if and only if there exist $s, s' \in S$ such that $\text{dist}(s, s') < \delta$.

$$\sqrt{\left(\frac{\delta}{2}\right)^2 + \left(\frac{\delta}{2}\right)^2} = \sqrt{\frac{\delta^2}{2}}$$

$$= \frac{\delta}{\sqrt{2}} < \delta$$

$s \leftarrow$ smallest y -coordinate
in this δ -strip.

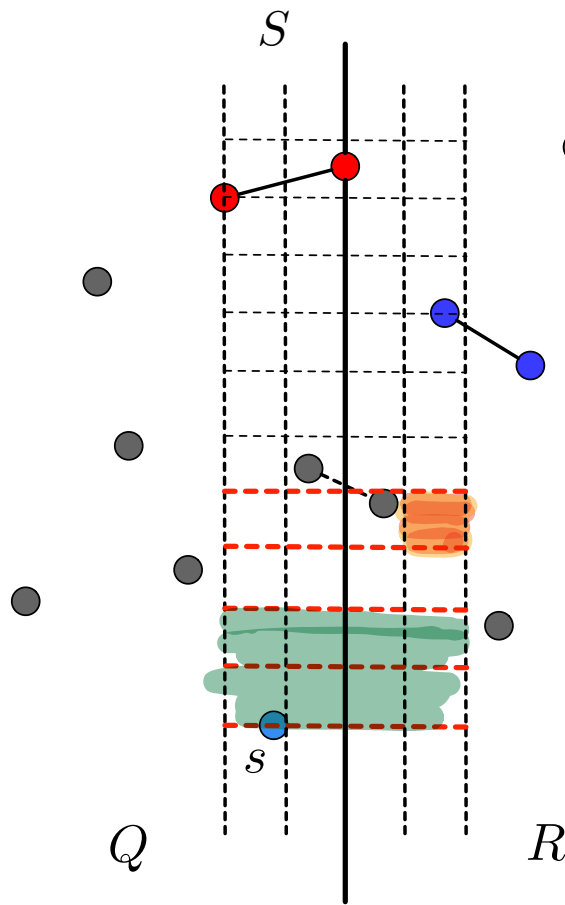
Closest Pair 2D

- **Lemma 2:** There exist $q \in Q$ and $r \in R$ for which $\text{dist}(q, r) < \delta$ if and only if there exist $s, s' \in S$ such that $\text{dist}(s, s') < \delta$.

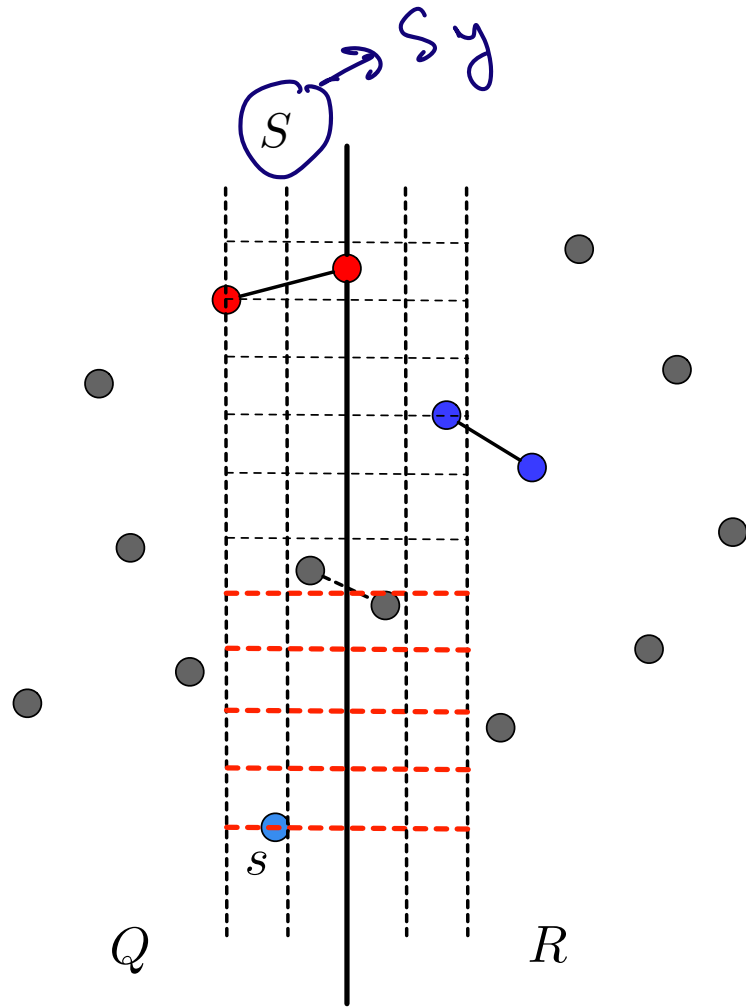
- **Lemma 3:** If $s, s' \in S$ has the property that $\text{dist}(s, s') < \delta$, then s' is located within 16 boxes from s .

Any point s' that is within distances $< \delta$ from s must lie in these lighted boxes.

One red-box can have at most 2 points.



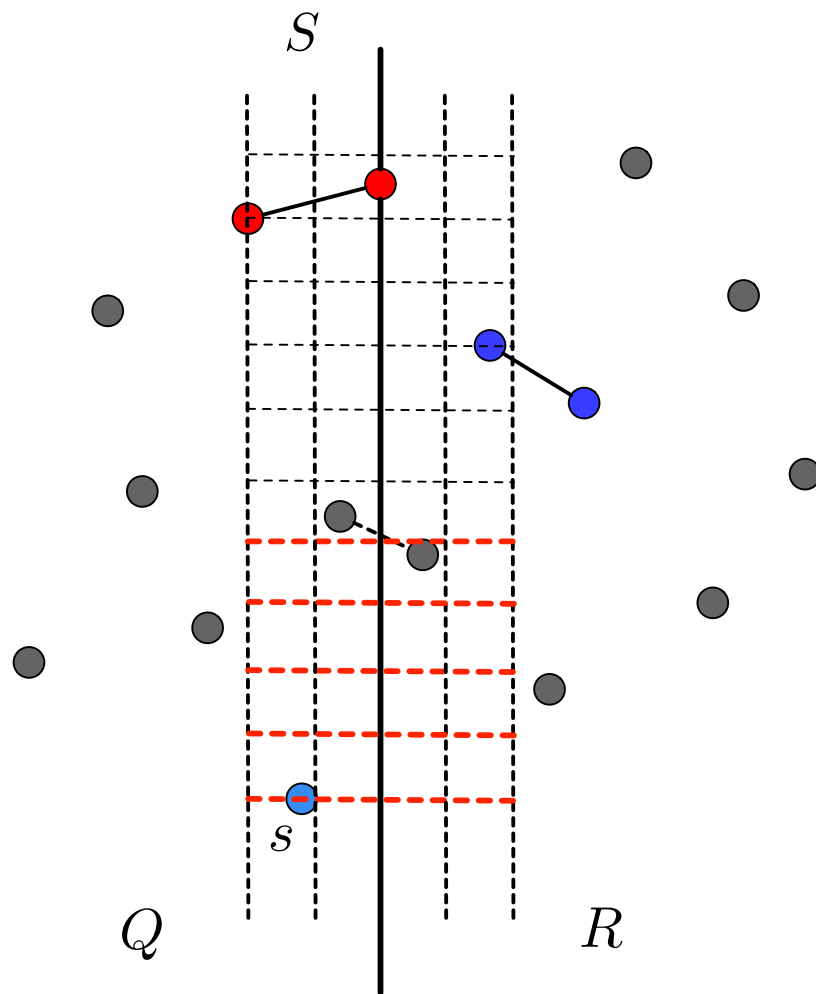
Closest Pair 2D



- **Lemma 3:** If $s, s' \in S$ has the property that $\text{dist}(s, s') < \delta$, then s' is located within 16 boxes from s .

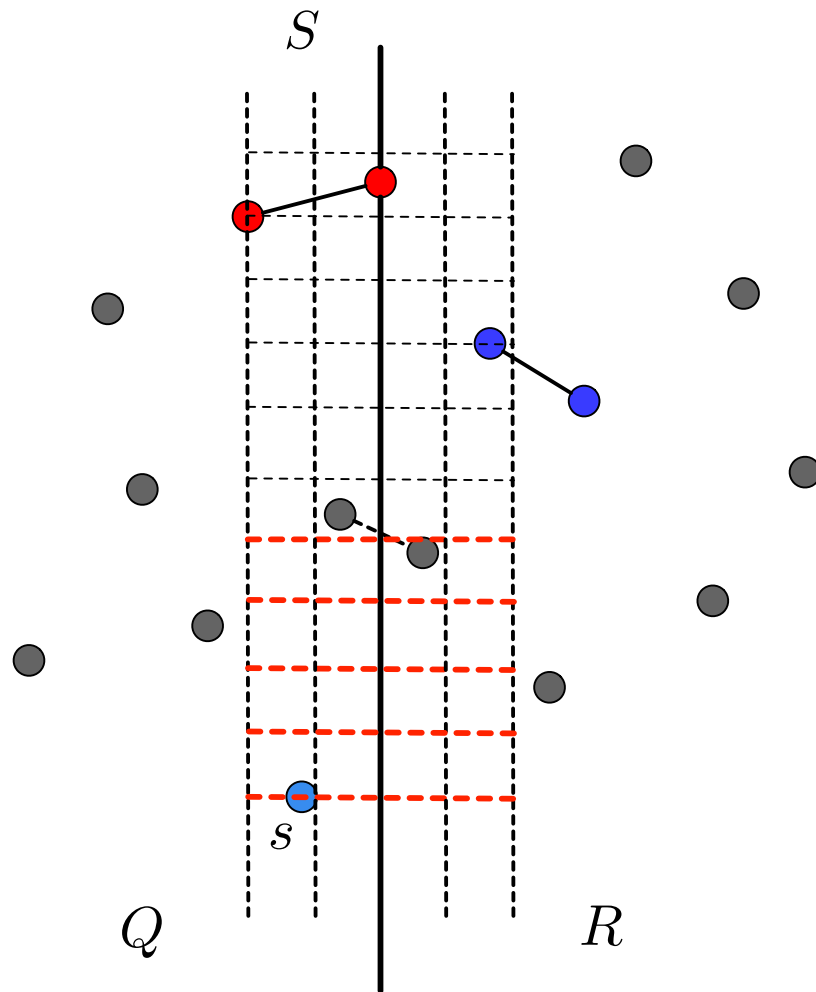
Get the points from Q_y and R_y (sorted order w.r.t. y coordinates).
that lie within the δ -strip.

How to compute S and S_y



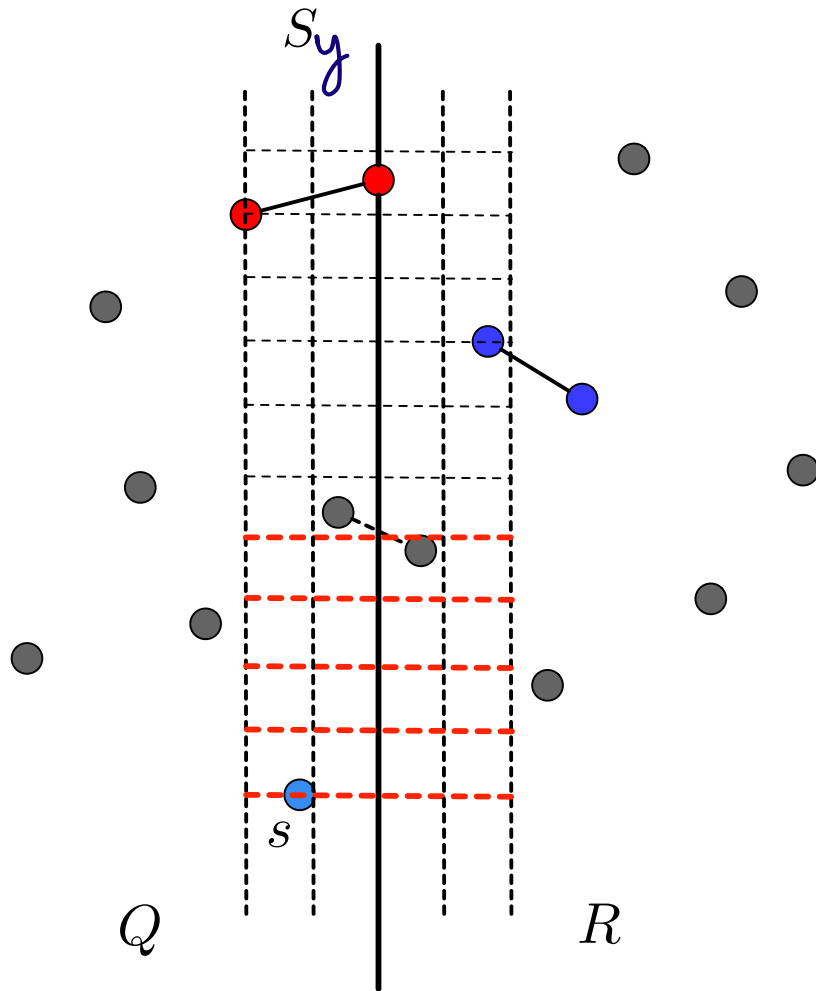
- P_y is the set of all points of P in sorted order of y -coordinates.

How to compute S and S_y



- P_y is the set of all points of P in sorted order of y -coordinates.
- Let $p = (p_x, p_y)$ be the point that lies on the median line.

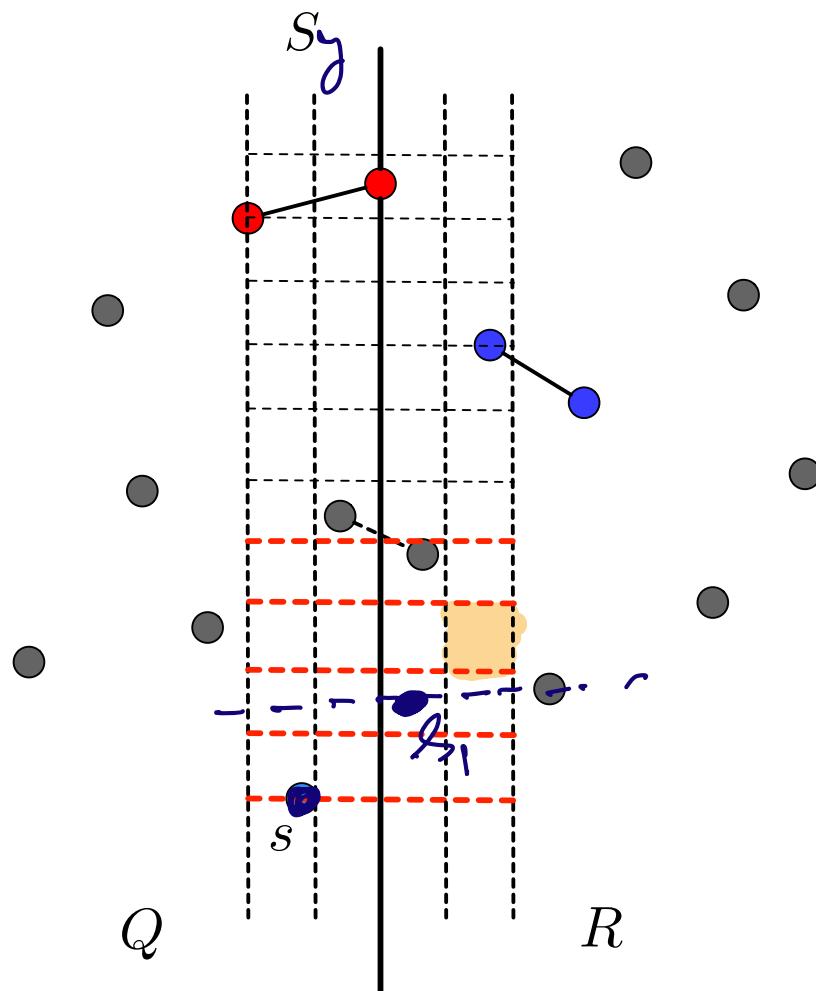
How to compute S and S_y



- P_y is the set of all points of P in sorted order of y -coordinates.
- Let $p = (p_x, p_y)$ be the point that lies on the median line.
- S is the set of points whose x -coordinate is at least $p_x - \delta$ and at most $p_x + \delta$.
- S_y is the set of these points from S in sorted order (can be computed from P_y).

Computing from $\mathcal{S}_y$

- S_y is the set of these points from S in sorted order (can be computed from P_y).

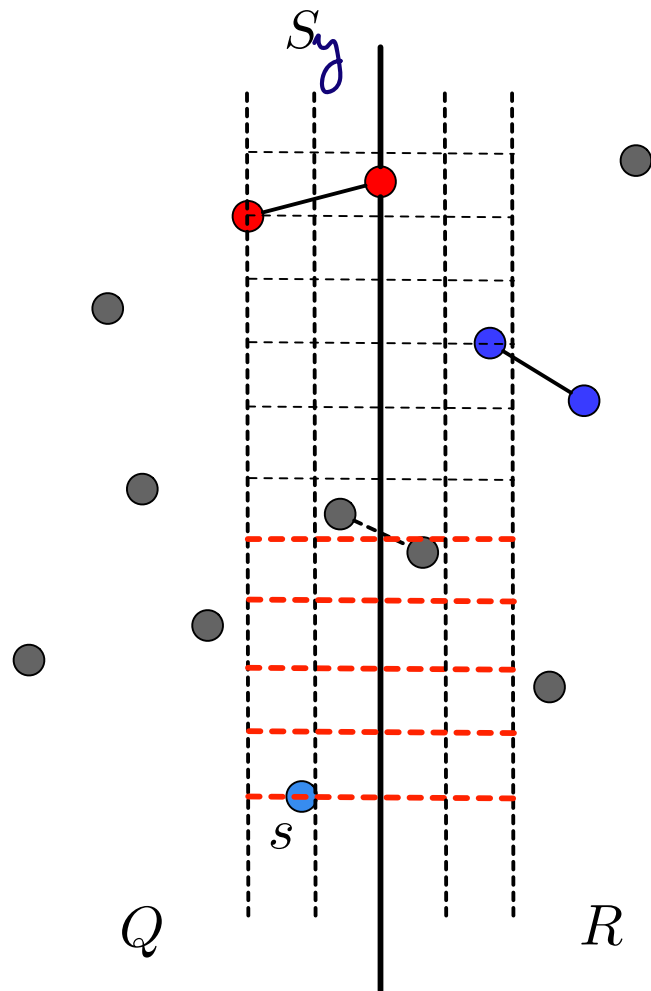


Compare the dist (s, f)
for all points f s.t.
 f lies within these
16 boxes.

there are $\leq (16 \times 2) = 32$
points.

$O(1)$

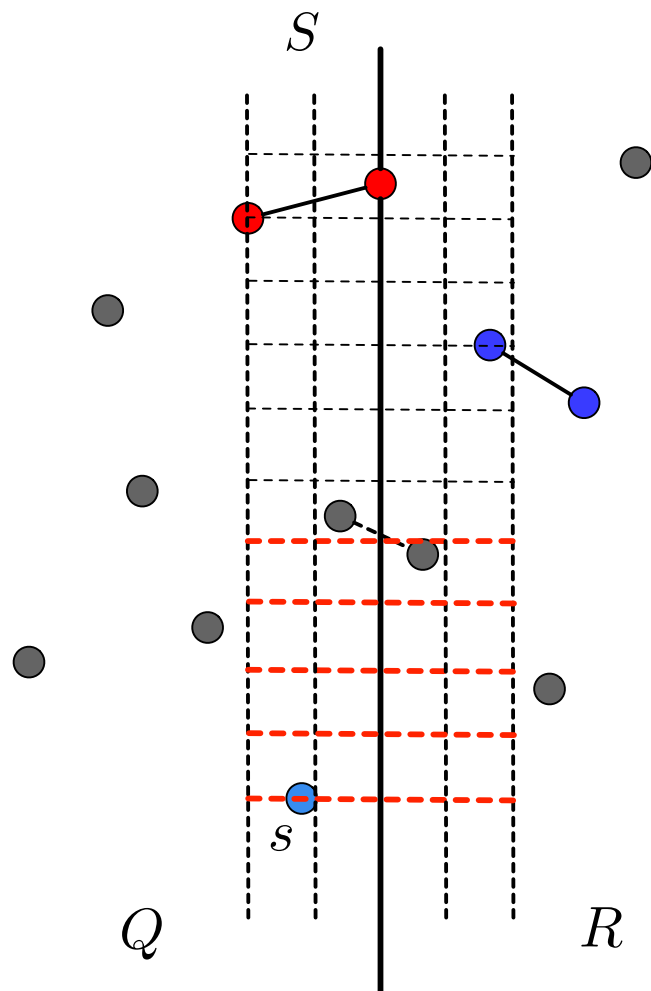
Computing from S_y



- S_y is the set of these points from S in sorted order (can be computed from P_y).
- For every $s \in S_y$, check with the next 16 points s' if $\text{dist}(s, s') < \delta$.
- If one such s' satisfying $\text{dist}(s, s') < \delta$ is found, then return $\text{dist}(s, s')$.
- Else, move to the next point in S_y and repeat the above procedure.

$O(n)$ -time.

Computing from S_y



- S_y is the set of these points from S in sorted order (can be computed from P_y).
- For every $s \in S_y$, check with the next 32 16 points s' if $\text{dist}(s, s') < \delta$.
- If one such s' satisfying $\text{dist}(s, s') < \delta$ is found, then return $\text{dist}(s, s')$.
- Else, move to the next point in S_y and repeat the above procedure.
- For example, let $S_y = \{s_1, s_2, \dots, s_{n'}\}$ (in sorted order of y -coordinates from S). Then, check
 $\text{dist}(s_1, s_2), \text{dist}(s_1, s_3), \dots, \text{dist}(s_1, s_{17}),$
 $\text{dist}(s_2, s_3), \text{dist}(s_2, s_4), \dots, \text{dist}(s_2, s_{18}), \dots$
(in this order).

Closet-Pair-Rec(P_x, P_y, n)

if $n \leq 3$ **then**

 | Solve by checking distance between all pairs;

end

else

 Compute Q_x, R_x, Q_y, R_y ;

$\delta_Q \leftarrow \text{Closet-Pair-Rec}(Q_x, Q_y, \lceil n/2 \rceil)$;

$\delta_R \leftarrow \text{Closet-Pair-Rec}(R_x, R_y, \lfloor n/2 \rfloor)$;

$S \leftarrow$ points within distance δ from median line L ;

 Compute S_y ;

for each point $s \in S_y$ **do**

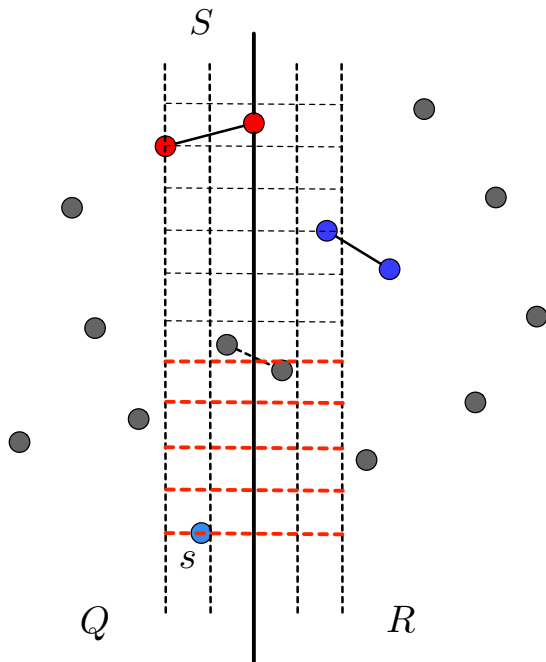
 | Compute distance to next 16 points from s ;

 | Let $\delta^* = \text{dist}(s, s')$ be the minimum of these distances;

end

 Output $\min\{\delta^*, \delta_Q, \delta_R\}$;

end



Analysis of Running Time

$\left\{ \begin{array}{l} P_x \leftarrow \text{sort the points of } P \text{ by } x\text{-coordinate;} \\ P_y \leftarrow \text{sort the points of } P \text{ by } y\text{-coordinate;} \\ \delta \leftarrow \text{Closest-Pair-Rec}(P_x, P_y, n); \end{array} \right. \rightarrow \begin{array}{l} O(n \log n) \\ \text{Preprocessing} \end{array}$

$$T(n) = 2T\left(\frac{n}{2}\right) + cn$$

$$\approx O(n \log n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n)$$

$$\underline{O(n \log^2 n)}$$

Analysis of Running Time

$P_x \leftarrow$ sort the points of P by x -coordinate;

$P_y \leftarrow$ sort the points of P by y -coordinate;

$\delta \leftarrow \text{Closet-Pair-Rec}(P_x, P_y, n);$

- Two different sorting operations $\mathcal{O}(n \log n)$ -time each.
- Then, a function CLOSEST-PAIR-REC that makes recursive call.

Analysis (continued)

Closet-Pair-Rec(P_x, P_y, n)

if $|P| \leq 3$ **then**

| *Solve by checking distance between all pairs;*

end

else

| *Compute Q_x, R_x, Q_y, R_y ;*

| $\delta_Q \leftarrow \text{Closet-Pair-Rec}(Q_x, Q_y, \lceil n/2 \rceil)$;

| $\delta_R \leftarrow \text{Closet-Pair-Rec}(R_x, R_y, \lfloor n/2 \rfloor)$;

| $S \leftarrow$ *points within distance δ from median line L ; Compute S_y ;*

| **for** *each point $s \in S_y$* **do**

| | *Compute distance to next 16 points from s ;*

| | *Let $\delta^* = \text{dist}(s, s')$ be the minimum of these distances;*

| **end**

| *Output $\min\{\delta^*, \delta_Q, \delta_R\}$;*

end

- Recurrence for Closest-Pair-Rec is $T(n) = 2T(n/2) + cn$.

References

- Chapter 5.4 from *Algorithm Design* by Kleinberg and Tardos book.